

Sistema de Análise Automática de Objetos em uma Imagem

Nome do aluno 1

Nome do aluno 2

Nome do aluno 3

Ciência da Computação — Universidade Tuiuti do Paraná

Processamento de Imagens — Prof. Diógenes Furlan

Avaliação Bimestral — 1º Bimestre — 2026

Resumo

Este trabalho apresenta um sistema capaz de analisar automaticamente uma imagem digital contendo um ou mais objetos, separando-os do fundo, identificando cada um individualmente e calculando suas características geométricas. O sistema foi implementado em C++ com OpenGL/GLUT e encadeia técnicas básicas de processamento digital de imagens: conversão para tons de cinza, filtragem espacial, análise de histograma, limiarização automática por Otsu, morfologia matemática e rotulação de componentes conexos. Para cada objeto identificado são calculadas área, centroide, *bounding box*, perímetro e circularidade. O sistema foi testado em três imagens de dificuldade crescente e identificou corretamente todos os objetos nas três.

Sumário

1	Introdução	2
2	Fundamentação Teórica	2
2.1	Histograma	2
2.2	Filtragem espacial	2
2.3	Limiarização	3
2.4	Morfologia matemática	3
2.5	Componentes conexos	4
2.6	Características geométricas	4
3	Metodologia	5
4	Implementação	6
4.1	Estrutura dos arquivos	6
4.2	Estruturas de dados	6
4.3	Principais funções	7
4.4	Dois pontos de implementação que merecem destaque	7
4.5	Interface	8

5	Resultados	8
5.1	Imagem 1 — situação simples (Circulos4.bmp)	8
5.2	Imagem 2 — com ruído (Circulos4-ruído5.bmp)	9
5.3	Imagem 3 — situação complexa (C1.bmp)	9
5.4	Análise de histograma	10
6	Discussão	11
7	Conclusão	13
	Referências	13

1 Introdução

O problema tratado neste trabalho é o da **análise automática de objetos** em uma imagem digital. Dada uma imagem contendo um ou mais objetos sobre um fundo contrastante, o sistema deve determinar, sem intervenção do usuário, quantos objetos existem, onde cada um está e quais são suas características geométricas.

As imagens trabalhadas são arquivos BMP de 8 e 24 bits, convertidos internamente para tons de cinza. Três situações foram consideradas, conforme pedido no enunciado:

1. **Situação simples:** objetos bem separados e com alto contraste em relação ao fundo;
2. **Presença de ruído:** a mesma imagem contaminada com ruído sal-e-pimenta, exigindo pré-processamento;
3. **Situação mais complexa:** objetos de tamanhos e formas diferentes, incluindo figuras vazadas (apenas o contorno).

O objetivo do trabalho é integrar as técnicas estudadas ao longo do bimestre, mostrando como algoritmos individualmente simples — uma contagem de pixels, uma comparação com um limiar, uma varredura de vizinhança — se combinam para resolver um problema de análise que nenhum deles resolveria sozinho.

Um objetivo secundário, que orientou várias decisões de projeto, foi tornar cada etapa **observável**: o programa permite executar o pipeline passo a passo, exibindo o resultado intermediário de cada operação, de modo que o efeito de cada técnica sobre a imagem possa ser visto e discutido isoladamente.

2 Fundamentação Teórica

2.1 Histograma

O histograma de uma imagem em tons de cinza é a contagem de quantos pixels existem para cada nível de intensidade. Para uma imagem de 8 bits, é um vetor de 256 posições em que $h[k]$ guarda o número de pixels com intensidade k :

$$h[k] = \#\{(x, y) \mid f(x, y) = k\}, \quad k = 0, 1, \dots, 255 \quad (1)$$

A soma de todas as posições é necessariamente o número total de pixels, $\sum_{k=0}^{255} h[k] = L \times C$. O histograma normalizado, obtido dividindo-se cada posição por $L \times C$, estima a probabilidade de um pixel qualquer ter determinada intensidade.

A distinção fundamental é que a **imagem** carrega informação espacial enquanto o **histograma** carrega informação estatística: a posição dos pixels é descartada. Duas imagens completamente diferentes podem ter histogramas idênticos. É justamente por ser estatístico que ele serve para escolher um limiar sem precisar saber onde os objetos estão.

2.2 Filtragem espacial

Filtros espaciais combinam a intensidade de um conjunto de pixels vizinhos para gerar a intensidade do pixel correspondente na imagem de saída. Foram implementados dois, com propósitos distintos.

Filtro de média. É um filtro passa-baixa, obtido pela convolução com uma máscara cujos coeficientes são positivos e somam 1:

$$w = \frac{1}{9} \begin{bmatrix} 1 & 1 & 1 \\ 1 & 1 & 1 \\ 1 & 1 & 1 \end{bmatrix} \quad g(x, y) = \sum_i \sum_j w(i, j) f(x + i, y + j) \quad (2)$$

Atenua as altas frequências — as transições abruptas — reduzindo ruído, mas borrando as bordas junto.

Filtro de mediana. Substitui cada pixel pela mediana das intensidades da vizinhança, isto é, pelo valor central da lista ordenada dos nove valores. É um filtro **não linear**: não há convolução de máscara, porque ordenar não é uma operação linear e nenhum conjunto de coeficientes o reproduz.

A vantagem sobre a média aparece no ruído impulsivo. Um pixel de valor extremo (0 ou 255) isolado em uma vizinhança homogênea vai parar em uma das pontas da lista ordenada e é simplesmente descartado, em vez de ser diluído na soma e contaminar o resultado — e, com ele, o dos oito vizinhos.

2.3 Limiarização

A limiarização converte a imagem de 256 níveis em uma imagem binária, comparando cada pixel com um valor de corte T :

$$b(x, y) = \begin{cases} 1 & \text{se } f(x, y) > T \quad (\text{objeto}) \\ 0 & \text{caso contrário} \quad (\text{fundo}) \end{cases} \quad (3)$$

É a operação que cria a marcação sobre a qual todas as etapas seguintes operam: morfologia, rotulação e medidas geométricas só existem sobre imagem binária.

Método de Otsu. Escolher T manualmente não é aceitável em um sistema automático. O método de Otsu percorre todos os 256 cortes possíveis e escolhe o que **maximiza a variância entre as duas classes**, o que é equivalente a minimizar a variância dentro de cada classe:

$$\sigma_b^2(T) = \omega_0(T) \omega_1(T) [\mu_0(T) - \mu_1(T)]^2 \quad (4)$$

onde ω_0 e ω_1 são as quantidades de pixels de cada classe e μ_0 e μ_1 suas intensidades médias. Intuitivamente: procura o corte que deixa os dois grupos mais separados entre si e mais homogêneos internamente. O cálculo usa apenas o histograma, sem varrer a imagem novamente.

2.4 Morfologia matemática

A morfologia matemática extrai informação sobre a geometria de um conjunto desconhecido transformando-o por meio de outro conjunto bem definido, o **elemento estruturante**. Neste trabalho o elemento estruturante é 3×3 preenchido, correspondente à vizinhança-8.

Erosão. $A \ominus B$ — o pixel permanece objeto apenas se *todos* os pixels sob o elemento estruturante forem objeto. É a operação AND da vizinhança. Contrai os objetos em uma camada, eliminando pontos isolados e saliências finas.

Dilatação. $A \oplus B$ — o pixel vira objeto se *algum* pixel sob o elemento estruturante for objeto. É a operação OR. Expande os objetos em uma camada, preenchendo buracos pequenos.

As duas são duais: o complemento de uma erosão é a dilatação do complemento pelo elemento estruturante refletido, $(A \ominus B)^c = A^c \oplus \hat{B}$.

Abertura e fechamento. São as composições das duas primitivas:

$$A \circ B = (A \ominus B) \oplus B \quad (\text{abertura}) \quad (5)$$

$$A \bullet B = (A \oplus B) \ominus B \quad (\text{fechamento}) \quad (6)$$

Na abertura, a erosão apaga o ruído e a dilatação seguinte devolve aos objetos sobreviventes o tamanho original — limpa sem encolher. No fechamento, a dilatação tapa buracos e fendas e a erosão recupera o contorno externo. A ordem importa: as duas operações não são comutativas, e cada uma resolve um problema para o qual a outra é cega.

2.5 Componentes conexos

Dois pixels p e q pertencentes ao conjunto S são conexos se existe um caminho entre eles formado inteiramente por pixels de S . O conjunto de todos os pixels conexos a um dado pixel é um **componente conexo**. Rotular os componentes é o que permite contar objetos e tratá-los individualmente.

A definição depende da relação de adjacência adotada:

- **Adjacência-4:** vizinhos horizontais e verticais, $N_4(p) = \{(x \pm 1, y), (x, y \pm 1)\}$;
- **Adjacência-8:** os quatro anteriores mais os quatro diagonais, $N_8(p) = N_4(p) \cup N_D(p)$.

A escolha altera o resultado: uma mesma figura pode apresentar 4 regiões por adjacência-8 e 10 por adjacência-4, porque nesta última objetos ligados apenas pela diagonal são contados separadamente. Este trabalho usa **adjacência-8**, que corresponde melhor à noção intuitiva de objeto.

2.6 Características geométricas

Para cada componente rotulado com o identificador r , sendo $R = \{(x, y) \mid \text{rótulo}(x, y) = r\}$:

Área $A = |R|$, o número de pixels do objeto.

Centroide as coordenadas médias dos pixels do objeto:

$$\bar{x} = \frac{1}{A} \sum_{(x,y) \in R} x \quad \bar{y} = \frac{1}{A} \sum_{(x,y) \in R} y \quad (7)$$

Bounding box o menor retângulo que envolve o objeto, $(x_{\min}, y_{\min})$ a $(x_{\max}, y_{\max})$.

Perímetro estimado pela contagem dos pixels de contorno, isto é, pixels do objeto que possuem ao menos um vizinho-4 fora dele.

Circularidade razão adimensional que compara a área com o quadrado do perímetro:

$$C = \frac{4\pi A}{P^2} \quad (8)$$

Quanto mais compacta a forma, maior o valor. Serve para comparar formas entre si.

3 Metodologia

A sequência de processamento adotada é a seguinte:

leitura do BMP → conversão para tons de cinza → filtro de mediana 3×3 → histograma → limiarização por Otsu → abertura + fechamento → rotulação (8-conexa) → filtro de área → análise geométrica

A ordem difere do diagrama sugerido no enunciado em dois pontos, ambos deliberados e justificados a seguir.

A morfologia vem antes da rotulação, não depois. O diagrama do enunciado posiciona os filtros morfológicos após a etapa de componentes conexos. Optamos pela ordem inversa porque o propósito da morfologia neste pipeline é *limpar* a imagem binária, e limpá-la depois de contar os objetos não altera a contagem já realizada — que é o resultado que o trabalho pede. Aplicada antes, a abertura elimina os pontos de ruído que de outro modo seriam contados como objetos. A Seção 6 apresenta a medição que sustenta essa escolha.

O histograma é calculado antes da filtragem e novamente depois. O histograma tem dois papéis distintos: é instrumento de diagnóstico, usado para entender a distribuição de intensidades da imagem e decidir se a limiarização é viável, e é insumo do método de Otsu. No primeiro papel interessa o histograma da imagem original; no segundo, o da imagem já filtrada, que é sobre a qual o limiar será aplicado.

Filtro de área. Após a rotulação, componentes com área inferior a um limite (50 pixels, ajustável em tempo de execução) são descartados e os rótulos restantes são reenumerados. É a última barreira contra falsos objetos: fragmentos de ruído que sobrevivem à filtragem e à morfologia são pequenos demais para serem objetos reais.

Deteção automática de polaridade. A limiarização precisa saber de que lado do limiar estão os objetos, e isso varia conforme a imagem: em `Circulos4.bmp` os objetos são claros sobre fundo escuro, e em `C1.bmp` são escuros sobre fundo claro. O sistema decide sozinho contando os pixels de cada lado do limiar e tomando como objeto a **classe minoritária**, partindo do princípio — declarado no enunciado — de que os objetos aparecem sobre um fundo contrastante e ocupam menos área que ele. Sem esse mecanismo, metade das imagens de teste seria processada com o fundo rotulado como um único objeto cobrindo a imagem inteira.

4 Implementação

O sistema foi escrito em C++ sobre o *framework* fornecido na disciplina (OpenGL com GLUT para exibição e uma biblioteca de leitura e escrita de BMP), portado de Windows/Code::Blocks para Linux com g++ e `freeglut`. As alterações necessárias ao porte estão documentadas em `codigo/PORTE.md`.

4.1 Estrutura dos arquivos

Arquivo	Responsabilidade
<code>src/main.cpp</code>	janela, teclado, menu e roteiros de demonstração
<code>src/pdi.cpp/.h</code>	todos os algoritmos do pipeline
<code>src/image_class.cpp/.h</code>	classe de imagem: pixels, exibição, arquivo
<code>src/bmp_lib2.cpp/.h</code>	leitura e escrita de arquivos BMP

Tabela 1: Organização do código-fonte.

4.2 Estruturas de dados

A decisão de projeto mais importante foi não operar diretamente sobre os pixels RGB da `ImageClass` a cada etapa. Em vez disso, a classe `PDI` mantém buffers próprios, vetores planos de $W \times H$ posições, indexados sempre da mesma forma:

```
1 int W, H; // dimensoes da imagem atual
2 vector<unsigned char> cinza; // 0..255
3 vector<unsigned char> bin; // 0 = fundo, 1 = objeto
4 vector<int> rotulo; // 0 = fundo, 1..n = objetos
5 int histograma[256];
6 vector<Objeto> objetos;
```

O acesso é sempre `buf[y*W + x]`. A uniformidade elimina a principal fonte de erro do trabalho — confundir linha com coluna — e mantém os dados contíguos em memória.

As características de cada objeto são guardadas em um `struct`:

```
1 struct Objeto {
2     int id, area;
3     double cx, cy; // centroide
4     int xmin, ymin, xmax, ymax; // bounding box
5     int perimetro;
6     double circularidade;
7 };
```

4.3 Principais funções

Função	O que faz
<code>paraCinza()</code>	converte a imagem de entrada para o buffer cinza
<code>calculaHistograma()</code>	conta pixels por nível de intensidade
<code>filtroMedia3x3()</code>	convolução com máscara de média
<code>filtroMediana3x3()</code>	mediana da vizinhança 3×3
<code>otsu()</code>	devolve o limiar ótimo a partir do histograma
<code>polaridadeAutomatica()</code>	decide de que lado do limiar está o objeto
<code>limiariza(T, claro)</code>	preenche bin
<code>erosao()</code> , <code>dilatacao()</code>	primitivas morfológicas
<code>abertura()</code> , <code>fechamento()</code>	composições das primitivas
<code>rotulaComponentes()</code>	rotula os componentes conexos; devolve a contagem
<code>floodFill()</code>	preenchimento por inundação, 8-conexo
<code>analisaObjetos(areaMin)</code>	calcula as medidas e aplica o filtro de área
<code>imprimeRelatorio()</code>	imprime a tabela no console
<code>desenhaObjetos()</code>	imagem rotulada com cores, caixas e centroides
<code>pipelineCompleto()</code>	encadeia tudo e grava as imagens intermediárias

Tabela 2: Principais funções da classe PDI.

4.4 Dois pontos de implementação que merecem destaque

Filtros e morfologia leem de uma cópia. Toda operação de vizinhança lê de uma cópia do buffer e escreve no buffer original. Se lesse e escrevesse no mesmo vetor, o pixel já processado contaminaria o cálculo do vizinho seguinte, produzindo um arrasto direcional no resultado.

O *flood fill* é iterativo, não recursivo. A versão recursiva apresentada em aula abre uma chamada de função por pixel visitado. Medimos experimentalmente o limite: a versão recursiva sobrevive a um objeto de 400×400 pixels, mas estoura a pilha em 512×512 — exatamente o tamanho do exemplo citado no enunciado. A implementação usa uma fila explícita (`std::queue`), que aloca no *heap* e não tem esse teto:

```
1 void PDI::floodFill(int x0, int y0, int rot)
2 {
3     queue< pair<int,int> > fila;
4     rotulo[y0*W + x0] = rot;
5     fila.push(make_pair(x0, y0));
6
7     while(!fila.empty()) {
8         int x = fila.front().first;
9         int y = fila.front().second;
10        fila.pop();
11
12        for(int dy = -1; dy <= 1; dy++)           // conectividade-8
13            for(int dx = -1; dx <= 1; dx++) {
14                int nx = x + dx, ny = y + dy;
15                if(nx < 0 || ny < 0 || nx >= W || ny >= H) continue;
16                if(bin[ny*W + nx] == 1 && rotulo[ny*W + nx] == 0) {
17                    rotulo[ny*W + nx] = rot;
18                    fila.push(make_pair(nx, ny));
19                }
20            }
21    }
```

```

19         }
20     }
21 }
22 }

```

4.5 Interface

Além da execução do pipeline completo em uma única ação, o programa oferece um menu com cinco **roteiros de demonstração** que percorrem o pipeline etapa por etapa, avançando sob comando do usuário (teclas N e P) e exibindo na própria janela a descrição da etapa corrente e os números calculados. As imagens intermediárias podem ser gravadas em disco em uma única ação.

5 Resultados

Os testes foram realizados em três imagens. Em todas, o limiar foi obtido automaticamente por Otsu, a polaridade foi detectada automaticamente e a área mínima foi fixada em 50 pixels.

5.1 Imagem 1 — situação simples (Circulos4.bmp)

Imagem de 400×400 pixels com quatro círculos claros bem separados sobre fundo preto. Limiar de Otsu: $T = 0$. Polaridade detectada: objeto claro.

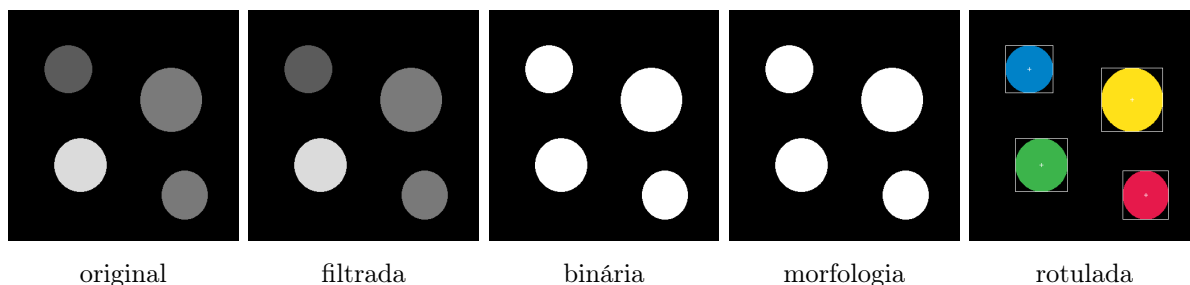


Figura 1: Etapas do processamento de *Circulos4.bmp*.

Objeto	Área	Centroide	Bounding box	Perím.	Circ.
1	5329	(306, 79)	(266,37)–(345,121)	230	1,266
2	6621	(125, 131)	(80,85)–(170,177)	257	1,260
3	9320	(282, 244)	(229,189)–(335,299)	307	1,243
4	5391	(104, 297)	(63,256)–(145,338)	232	1,259

Tabela 3: Objetos identificados em *Circulos4.bmp*.

Os quatro objetos foram identificados corretamente. As circularidades são praticamente idênticas entre si, como esperado de quatro círculos, e as áreas diferem porque os raios são diferentes.

5.2 Imagem 2 — com ruído (Circulos4-ruído5.bmp)

A mesma imagem anterior contaminada com ruído sal-e-pimenta em 5% dos pixels, gerada pelo próprio programa. Gerar a imagem por código, em vez de procurar uma pronta, permitiu controlar a intensidade do ruído e dispor da imagem limpa correspondente para comparação. Limiar de Otsu: $T = 0$.

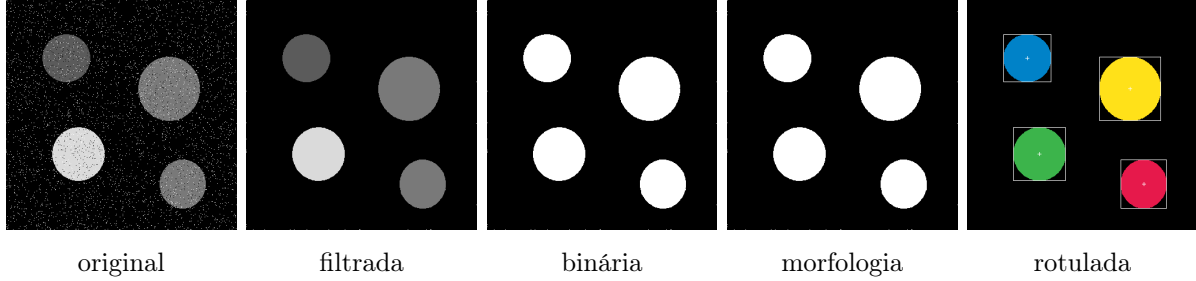


Figura 2: Etapas do processamento de `Circulos4-ruído5.bmp`. A diferença entre a primeira e a segunda imagem mostra a ação do filtro de mediana.

Objeto	Área	Centroide	Bounding box	Perím.	Circ.
1	5335	(306, 79)	(266,37)–(345,121)	230	1,267
2	6611	(125, 131)	(80,85)–(170,177)	258	1,248
3	9323	(282, 244)	(229,189)–(335,299)	307	1,243
4	5390	(104, 297)	(63,256)–(145,338)	233	1,248

Tabela 4: Objetos identificados em `Circulos4-ruído5.bmp`.

Comparando com a Imagem 1, as áreas diferem em no máximo 10 pixels (0,15%), os centroides são idênticos e as *bounding boxes* coincidem. O ruído foi integralmente absorvido pelo pré-processamento.

5.3 Imagem 3 — situação complexa (C1.bmp)

Imagem de 512×384 pixels com cinco figuras geométricas de tamanhos e formas diferentes, escuras sobre fundo claro. Três são preenchidas e duas são vazadas — apenas o contorno. Limiar de Otsu: $T = 192$. Polaridade detectada: objeto escuro.

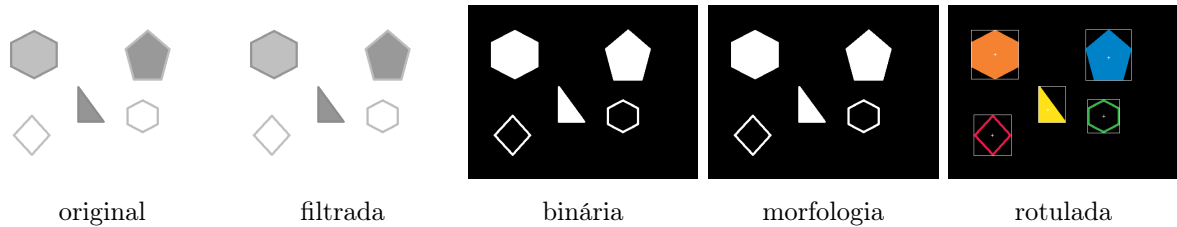


Figura 3: Etapas do processamento de `C1.bmp`.

Objeto	Área	Centroide	Bounding box	Perím.	Circ.
1	1167	(98, 96)	(57,51)–(140,141)	344	0,124
2	1106	(345, 138)	(309,101)–(380,175)	400	0,087
3	2717	(221, 152)	(200,124)–(260,204)	220	0,705
4	8114	(356, 267)	(305,217)–(406,329)	304	1,103
5	8728	(104, 274)	(51,220)–(156,328)	318	1,085

Tabela 5: Objetos identificados em `C1.bmp`.

Esta é a imagem mais informativa quanto à circularidade. Os objetos 4 e 5 são figuras preenchidas e compactas, com $C > 1,0$. O objeto 3 é um triângulo preenchido, forma menos compacta, com $C = 0,705$. Os objetos 1 e 2 são as figuras **vazadas**: têm área pequena e perímetro enorme, porque o contorno tem duas bordas — externa e interna — e a circularidade despenca para 0,124 e 0,087. A medida separa com clareza três categorias de forma.

5.4 Análise de histograma

A Figura 4 compara os histogramas de duas imagens de naturezas opostas e explica por que a limiarização funciona em uma e não na outra.

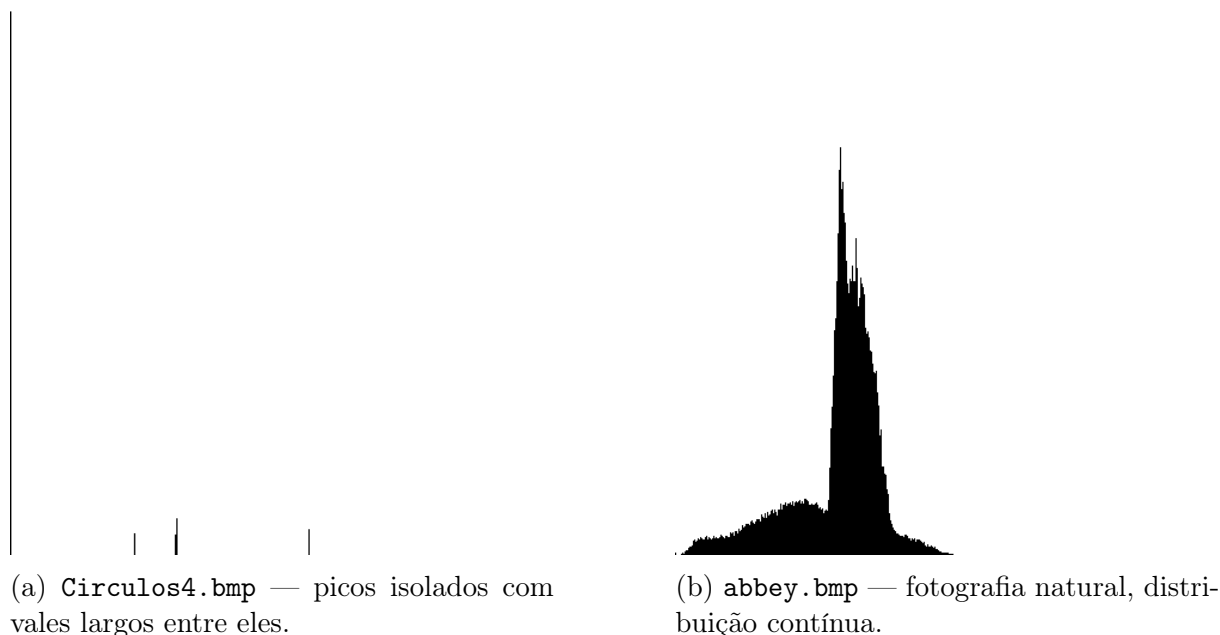


Figura 4: Histogramas de uma imagem sintética e de uma fotografia.

Em `Circulos4.bmp` os pixels se concentram em poucos níveis (0 para o fundo, 122 e 218 para os objetos), separados por faixas vazias. Qualquer limiar dentro de uma dessas faixas produz o mesmo resultado. Em `abbey.bmp`, uma fotografia, o pico estreito corresponde ao céu — uma região extensa e quase uniforme — e a base larga à alvenaria texturizada; não existe vale, e nenhum limiar separa os dois.

6 Discussão

O método conseguiu identificar todos os objetos? Sim, nas três imagens. Quatro objetos em `Circulos4.bmp`, quatro na versão ruidosa e cinco em `C1.bmp`, todos conferindo com a contagem visual.

Houve falsos objetos? Não no resultado final, mas houve em abundância nas etapas intermediárias. A Tabela 6 mostra a contagem de componentes na imagem ruidosa conforme as etapas de limpeza são ativadas.

Configuração	Componentes
Sem mediana, sem morfologia, sem filtro de área	2975
Sem mediana, com morfologia, sem filtro de área	40
Com mediana, sem morfologia, sem filtro de área	40
Com mediana, com morfologia, sem filtro de área	40
Com mediana, com morfologia, área ≥ 50 px	4

Tabela 6: Componentes detectados em `Circulos4-ruido5.bmp` conforme as etapas de limpeza. O valor correto é 4.

Sem nenhum pré-processamento, o sistema encontraria 2975 objetos onde existem 4: quase todo pixel de ruído isolado vira um componente. Vale notar que mediana e morfologia são *alternativas* nesta imagem — cada uma sozinha reduz de 2975 para 40 — e que o filtro de área é o que fecha a conta.

Algum objeto foi perdido? Nenhum. A comparação entre as Tabelas de `Circulos4.bmp` e de sua versão ruidosa mostra que nem mesmo as medidas foram afetadas de forma significativa: a maior diferença de área é de 10 pixels em 6621, ou 0,15%.

A escolha do limiar foi adequada? Sim, e o histograma explica por quê. Em `Circulos4.bmp` existe uma faixa larga de limiares que produzem o mesmo resultado — qualquer valor entre 1 e 121 separa igualmente bem, pois não há pixel algum nessa faixa. O método de Otsu não precisou ser preciso, apenas cair dentro do vale. Em `C1.bmp`, com $T = 192$, o resultado também foi correto.

A robustez, porém, é propriedade da imagem, não do método. Testamos o mesmo pipeline em `abbey.bmp`, uma fotografia, e nenhum limiar separa a edificação do céu, porque o histograma não tem vale (Figura 4). A limiarização global pressupõe o fundo contrastante que o enunciado estabelece.

A morfologia melhorou o resultado? Sim, e de forma mensurável: na Tabela 6, sozinha ela reduz a contagem de 2975 para 40.

Mais interessante é o *tipo* de morfologia. Comparamos a abertura com a erosão pura sobre a mesma imagem binária: a erosão remove o ruído, mas continua consumindo os objetos a cada aplicação — medimos um quadrado de 400 pixels caindo para 315 após uma erosão e para 231 após duas. A abertura remove o mesmo ruído e devolve o tamanho: 399 pixels após uma aplicação, e 399 novamente após a segunda, isto é, a operação é idempotente. É essa propriedade que a torna segura no pipeline.

O ruído afetou a identificação? Não no resultado final, mas afetaria drasticamente sem pré-processamento, como mostra a Tabela 6. Quanto à escolha do filtro, medimos a diferença entre média e mediana sobre uma imagem sintética com 5% de ruído: a mediana reconstrói a imagem original com erro médio de 0,05 níveis de cinza, contra 6,90 da média — 138 vezes maior. O perfil atravessando uma aresta mostra a razão:

Coluna	37	38	39	40	41	42	43
Original	40	40	40	200	200	200	200
Mediana	40	40	40	200	200	200	200
Média	40	40	93	146	200	200	200

Tabela 7: Perfil de intensidade atravessando a borda de um quadrado. A mediana preserva o degrau; a média o transforma em rampa.

A média não remove o ruído impulsivo, ela o *espalha*: um pixel branco isolado vira uma mancha 3×3 . A mediana o descarta e ainda preserva a borda intacta. Por isso a mediana foi adotada no pipeline.

Quais limitações foram observadas?

1. **A morfologia não limpa a borda da imagem.** Como o elemento estruturante 3×3 não cabe sobre os pixels da moldura, esses pixels são preservados sem modificação. Verificamos que, dos 36 fragmentos de ruído que sobrevivem à abertura em `Circulos4-ruído5.bmp`, **todos os 36 estão encostados na borda** e nenhum no interior. O filtro de área remove esses restos, mas a limitação existe.
2. **O perímetro é subestimado, e a circularidade sai acima de 1.** Contar pixels de contorno subestima o comprimento real, porque em um trecho diagonal cada pixel avança $\sqrt{2}$ mas conta como 1. Medimos a razão em círculos digitais de três raios diferentes (10, 20 e 40): o perímetro contado é sempre 0,891 do valor teórico $2\pi r$. Como P entra ao quadrado no denominador da Equação 8, a circularidade de um círculo sai em torno de 1,26 em vez de 1,0. Isso não invalida a medida — a ordem entre as formas se mantém, que é o uso pedido — mas o valor absoluto não deve ser lido como se 1,0 fosse o círculo perfeito.
3. **A limiarização é global.** Um único limiar para a imagem inteira não lida com iluminação não uniforme. Limiarização adaptativa resolveria, ao custo de complexidade.
4. **Objetos encostados são contados como um só.** A rotulação não tem como separar dois objetos que se tocam; seriam necessárias técnicas como a transformada de distância seguida de *watershed*.
5. **A detecção de polaridade pressupõe objetos minoritários.** Se os objetos ocupassem mais da metade da imagem, a heurística inverteria a classificação. O sistema permite sobrepor a decisão manualmente.

7 Conclusão

O sistema atingiu o objetivo proposto: identificou corretamente todos os objetos nas três imagens de teste, incluindo a versão contaminada com ruído, e calculou cinco características geométricas para cada um deles. O trabalho mostrou que a combinação de algoritmos simples — contagem de pixels, comparação com limiar, operações lógicas sobre vizinhança e busca em largura — resolve um problema de análise que nenhum deles resolveria isoladamente.

A lição mais clara dos experimentos é que **o pré-processamento não é acessório**. A diferença entre 2975 falsos objetos e os 4 corretos não está no algoritmo de rotulação, que é o mesmo nos dois casos, mas nas etapas de limpeza que o antecedem. Do mesmo modo, a escolha entre média e mediana, que parece um detalhe, define se as bordas dos objetos sobrevivem ao pré-processamento.

A segunda lição é que a qualidade do resultado depende tanto da imagem quanto do método. A limiarização global funcionou porque as imagens de teste têm histograma com vales largos; na fotografia testada, nenhum limiar funciona. Reconhecer essa dependência é parte de aplicar a técnica corretamente.

Possíveis melhorias.

- **Limiarização adaptativa**, calculando um limiar por região, para tolerar iluminação não uniforme;
- **Perímetro por rastreamento de contorno**, contabilizando $\sqrt{2}$ nos passos diagonais, o que corrigiria a escala da circularidade;
- **Separação de objetos encostados** por transformada de distância e *watershed*;
- **Tratamento das bordas** na morfologia, replicando os pixels da moldura em vez de preservá-los;
- **Descritores adicionais** — excentricidade, momentos de Hu, número de Euler — que permitiriam classificar os objetos por forma, e não apenas medi-los.

Referências

Referências

- [1] GONZALEZ, R. C.; WOODS, R. E. **Processamento de Imagens Digitais**. São Paulo: Edgard Blücher, 2000.
- [2] CONCI, A.; AZEVEDO, E. **Computação Gráfica**. Volumes 1 e 2. Rio de Janeiro: Campus, 2007.
- [3] OTSU, N. A Threshold Selection Method from Gray-Level Histograms. **IEEE Transactions on Systems, Man, and Cybernetics**, v. 9, n. 1, p. 62–66, 1979.
- [4] FURLAN, D. **Notas de aula — Processamento de Imagens**. Universidade Tuiuti do Paraná, 2026.